

# ReconForge Module Documentation

Version 1.0 — Last updated: 2026-03-21

## Module Architecture Pattern

All modules follow the same layered pipeline:

```
tools/ → parsers/ → phases/ → module.py → core/
```

Each module has:

- A **base class** (ABC) defining the phase interface with 11 standard constructor parameters
- **Tool wrappers** that build `list[str]` commands via `Runner`
- **Parsers** that extract structured data from raw tool output
- **Phases** that orchestrate tool execution, parsing, and finding generation
- A **module orchestrator** that sequences phases and manages lifecycle

## Network Module ( `modules/network/` )

### Overview

Network-level reconnaissance: host discovery, port scanning, service enumeration, and authentication testing.

**Orchestrator:** `NetworkModule` ( `network_module.py` )

**Base Class:** `NetworkPhaseBase` ( `base.py` )

**CLI Phase Slugs:** `discovery`, `scanning`, `enumeration`, `authentication`

### Tools

| Tool       | Wrapper                          | Description                                                           |
|------------|----------------------------------|-----------------------------------------------------------------------|
| nmap       | <code>tools/nmap.py</code>       | Host discovery, port scanning, service/version detection, NSE scripts |
| enum4linux | <code>tools/enum4linux.py</code> | SMB/NetBIOS enumeration                                               |
| smbclient  | <code>tools/smbclient.py</code>  | SMB share access and listing                                          |
| ldapsearch | <code>tools/ldapsearch.py</code> | LDAP directory enumeration                                            |
| hydra      | <code>tools/hydra.py</code>      | Authentication brute-force testing ( <b>opt-in only</b> )             |

## Parsers

| Parser                            | Input             | Output                                 |
|-----------------------------------|-------------------|----------------------------------------|
| <code>nmap_parser.py</code>       | nmap stdout/XML   | Hosts, ports, services, versions       |
| <code>enum4linux_parser.py</code> | enum4linux stdout | Users, shares, groups, password policy |
| <code>smb_parser.py</code>        | smbclient stdout  | Shares, permissions, files             |
| <code>ldap_parser.py</code>       | ldapsearch stdout | LDAP entries, attributes               |

## Phases

| # | Internal<br>PHASE_NAME             | CLI Slug                    | Key Actions                                                              |
|---|------------------------------------|-----------------------------|--------------------------------------------------------------------------|
| 1 | <code>host_discovery</code>        | <code>discovery</code>      | Ping sweep, ARP scan, live host identification                           |
| 2 | <code>port_scanning</code>         | <code>scanning</code>       | SYN/connect scan, port enumeration, service detection                    |
| 3 | <code>service_enumeration</code>   | <code>enumeration</code>    | SMB enumeration, LDAP queries, deep service analysis                     |
| 4 | <code>authentication_checks</code> | <code>authentication</code> | Anonymous access testing, credential testing, hydra brute-force (opt-in) |

## Module-Specific Options

| CLI Flag                                 | Description                               |
|------------------------------------------|-------------------------------------------|
| <code>--brute-force</code>               | Enable hydra brute-force testing (opt-in) |
| <code>--phases discovery,scanning</code> | Run specific phases                       |

### `run()` Signature

```
NetworkModule.run(phases: Optional[List[str]] = None, brute_force: bool = False) -> Dict
```

---

## Web Module ( `modules/web/` )

---

### Overview

Web application reconnaissance: technology fingerprinting, content discovery, vulnerability scanning, and exploit candidate identification.

**Orchestrator:** `WebModule` ( `web_module.py` )

**Base Class:** `WebPhaseBase` ( `base.py` )

**Valid Phases:** `surface` , `content` , `vuln` , `exploit`

### Tools

| Tool      | Wrapper                         | Description                           |
|-----------|---------------------------------|---------------------------------------|
| whatweb   | <code>tools/whatweb.py</code>   | Web technology fingerprinting         |
| wafw00f   | <code>tools/wafw00f.py</code>   | WAF detection                         |
| nikto     | <code>tools/nikto.py</code>     | Web vulnerability scanning            |
| gobuster  | <code>tools/gobuster.py</code>  | Directory/file brute-forcing          |
| ffuf      | <code>tools/ffuf.py</code>      | Fast web fuzzing                      |
| wpscan    | <code>tools/wpscan.py</code>    | WordPress security scanning           |
| nuclei    | <code>tools/nuclei.py</code>    | Template-based vulnerability scanning |
| sqlmap    | <code>tools/sqlmap.py</code>    | SQL injection detection/exploitation  |
| curl_tool | <code>tools/curl_tool.py</code> | HTTP request utility                  |

## Parsers

| Parser                          | Input           | Output                             |
|---------------------------------|-----------------|------------------------------------|
| <code>gobuster_parser.py</code> | gobuster stdout | Discovered paths, status codes     |
| <code>ffuf_parser.py</code>     | ffuf JSON       | Fuzzed endpoints, responses        |
| <code>nikto_parser.py</code>    | nikto stdout    | Vulnerabilities, misconfigurations |
| <code>nuclei_parser.py</code>   | nuclei JSON     | Template matches, CVEs             |
| <code>whatweb_parser.py</code>  | whatweb JSON    | Technologies, versions             |
| <code>wafw00f_parser.py</code>  | wafw00f stdout  | WAF identification                 |
| <code>wpscan_parser.py</code>   | wpscan JSON     | WordPress vulnerabilities, plugins |

## Phases

| # | Internal<br>PHASE_NAME              | CLI Slug             | Key Actions                                                                     |
|---|-------------------------------------|----------------------|---------------------------------------------------------------------------------|
| 1 | <code>surface_discovery</code>      | <code>surface</code> | Technology fingerprinting, WAF detection, SSL testing                           |
| 2 | <code>content_enumeration</code>    | <code>content</code> | Directory brute-forcing, file discovery, vhost enumeration                      |
| 3 | <code>vulnerability_scanning</code> | <code>vuln</code>    | Nuclei templates, nikto scans, known CVE checks                                 |
| 4 | <code>exploit_candidates</code>     | <code>exploit</code> | SQLMap testing, exploit identification<br>( <b>opt-in via phase selection</b> ) |

### `run()` Signature

```
WebModule.run(phases: Optional[List[str]] = None, opt_in: bool = False) -> Dict
```

The `opt_in` parameter controls whether the `exploit` phase runs. When phases are specified via CLI, including `exploit` in the list automatically sets `opt_in=True`.

---

## API Module ( `modules/api/` )

---

### Overview

API security assessment: endpoint discovery, authentication testing (JWT, OpenAPI), parameter fuzzing, and authorization testing.

**Orchestrator:** `APIModule ( api_module.py )`

**Base Class:** `APIPhaseBase ( base.py )`

**Valid Phases:** `discovery , authentication , fuzzing , authorization`

### Tools

| Tool   | Wrapper                          | Description                                |
|--------|----------------------------------|--------------------------------------------|
| httpx  | <code>tools/httpx_tool.py</code> | Fast HTTP probing and technology detection |
| arjun  | <code>tools/arjun_tool.py</code> | HTTP parameter discovery                   |
| ffuf   | <code>tools/ffuf_api.py</code>   | API endpoint fuzzing                       |
| nuclei | <code>tools/nuclei_api.py</code> | API-specific vulnerability templates       |

### Parsers

| Parser                         | Input                               | Output                                                                                |
|--------------------------------|-------------------------------------|---------------------------------------------------------------------------------------|
| <code>openapi_parser.py</code> | OpenAPI 3.x / Swagger 2.x JSON/YAML | Endpoints, parameters, security schemes, <code>\$ref</code> resolution, JWT detection |
| <code>arjun_parser.py</code>   | arjun JSON                          | Discovered parameters                                                                 |
| <code>ffuf_parser.py</code>    | ffuf JSON                           | Fuzzed endpoints, responses                                                           |
| <code>nuclei_parser.py</code>  | nuclei JSON                         | API vulnerability matches                                                             |

## Phases

| # | Phase          | Key Actions                                                                          |
|---|----------------|--------------------------------------------------------------------------------------|
| 1 | discovery      | OpenAPI/Swagger spec detection, endpoint enumeration, httpx probing                  |
| 2 | authentication | JWT analysis via <code>is_jwt_bearer()</code> , token testing, auth scheme detection |
| 3 | fuzzing        | Parameter fuzzing, endpoint fuzzing, error-based detection                           |
| 4 | authorization  | IDOR testing, privilege escalation checks, role-based access testing                 |

## JWT & OpenAPI Features

- **OpenAPI Parser** handles both OpenAPI 3.x and Swagger 2.x with full `$ref` resolution
- **JWT Detection:** `is_jwt_bearer()` identifies Bearer tokens with JWT format in OpenAPI security schemes
- **Authorization Phase** explicitly uses `confidence="heuristic"` + `severity="low"` for pattern-based detections
- **Fuzzing Phase** documents that HTTP 500 alone is classified as `heuristic/info`

## `run()` Signature

```
APIModule.run(phases: Optional[List[str]] = None, opt_in: bool = False) -> Dict
```

The `opt_in` parameter controls whether the `authorization` phase runs. Including `authorization` in the CLI phases list automatically sets `opt_in=True`.

## Module-Specific Options

| CLI Flag                                  | Description                                  |
|-------------------------------------------|----------------------------------------------|
| <code>--auth-token "Bearer eyJ..."</code> | Bearer/auth token for authenticated requests |
| <code>--header "X-API-Key: abc"</code>    | Extra HTTP headers (repeatable)              |

## Surface Module ( `modules/surface/` )

### Overview

Attack surface mapping: correlates ports, services, and URLs into a unified attack surface map with confidence scoring and attack prioritization.

**Orchestrator:** `SurfaceModule` ( `surface_module.py` )

**Base Class:** `SurfacePhaseBase` ( `base.py` )

**Valid Phases:** `port_discovery` , `service_fingerprint` , `vector_correlation` , `prioritization`

## Architecture (Extended)

```
tools/ → parsers/ → intelligence/ → phases/ → surface_module.py → core/
```

The `intelligence/` layer is unique to the Surface module.

## Tools

| Tool                    | Wrapper                                | Description                                   |
|-------------------------|----------------------------------------|-----------------------------------------------|
| nmap (stealth profiles) | <code>tools/nmap_stealth.py</code>     | Port discovery with OPSEC-aware scan profiles |
| service_detector        | <code>tools/service_detector.py</code> | HTTP service probing                          |

## Intelligence Layer ( `intelligence/` )

| Component                            | Description                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>correlation_engine.py</code>   | Correlates ports/services/URLs into <code>CorrelatedService</code> entries → <code>AttackSurfaceMap</code>                                                                                                                                                                                                                                       |
| <code>confidence_scorer.py</code>    | Multi-signal scoring: <code>port_match</code> (0.25), <code>banner_match</code> (0.25), <code>version_detected</code> (0.20), <code>multi_detection</code> (0.20), <code>http_confirmed</code> (0.10) → labels: <code>confirmed</code> $\geq 0.80$ , <code>high</code> $\geq 0.60$ , <code>medium</code> $\geq 0.40$ , <code>low</code> $< 0.40$ |
| <code>deduplicator.py</code>         | <code>ServiceDeduplicator</code> merges duplicate detections from port scan and version scan phases                                                                                                                                                                                                                                              |
| <code>service_normalizer.py</code>   | Normalizes service names across tools                                                                                                                                                                                                                                                                                                            |
| <code>service_intelligence.py</code> | <code>ServiceIntelligenceDB</code> — enrichment data (categories, attack contexts, common tools, high-value flags)                                                                                                                                                                                                                               |
| <code>attack_prioritizer.py</code>   | Prioritizes attack vectors by risk                                                                                                                                                                                                                                                                                                               |

## Phases

| # | Phase               | Key Actions                                       |
|---|---------------------|---------------------------------------------------|
| 1 | port_discovery      | Stealth/normal/aggressive port scanning           |
| 2 | service_fingerprint | Version detection, banner grabbing                |
| 3 | vector_correlation  | Links ports/services/URLs into attack surface map |
| 4 | prioritization      | Ranks attack vectors by risk and confidence       |

### run() Signature

```
SurfaceModule.run(phases: Optional[List[str]] = None) -> Dict
```

## AD Module ( modules/ad/ )

### Overview

Active Directory reconnaissance: user/group enumeration, misconfiguration analysis, delegation discovery, attack path generation, and structured reporting.

**Orchestrator:** ADModule ( ad\_module.py )

**Base Class:** ADPhaseBase ( base.py )

**Valid Phases:** passive, identity, configuration, delegation, bloodhound

### Architecture (Extended)

```
tools/  parsers/  collectors/  analyzers/  attack_paths/  phases/  reporting/
ad_module.py  core/
```

No file exceeds ~250 lines. Clean separation across 8 sub-packages.



## Tools (8)

| Tool                                                  | Wrapper                                 | Description                                      |
|-------------------------------------------------------|-----------------------------------------|--------------------------------------------------|
| nmap                                                  | <code>tools/nmap.py</code>              | AD service port scanning, LDAP/SMB scripts       |
| enum4linux-ng                                         | <code>tools/enum4linux_ng.py</code>     | Next-gen SMB/MSRPC enumeration                   |
| Impacket (GetADUsers, GetNPUsers, lookupsid, rpcdump) | <code>tools/impacket.py</code>          | LDAP user dump, AS-REP roasting, SID brute-force |
| Impacket (findDelegation, GetUserSPNs)                | <code>tools/advanced_impacket.py</code> | Delegation discovery, Kerberoasting              |
| ldapsearch                                            | <code>tools/ldapsearch.py</code>        | LDAP directory queries                           |
| smbclient                                             | <code>tools/smbclient.py</code>         | SMB share enumeration                            |
| BloodHound                                            | <code>tools/bloodhound.py</code>        | AD graph data collection                         |
| NetExec                                               | <code>tools/netexec.py</code>           | Multi-protocol enumeration                       |

## Parsers (8)

| Parser                               | Description                   |
|--------------------------------------|-------------------------------|
| <code>nmap_parser.py</code>          | AD service scan results       |
| <code>enum4linux_ng_parser.py</code> | enum4linux-ng output          |
| <code>impacket_parser.py</code>      | Impacket tool output          |
| <code>smb_parser.py</code>           | SMB enumeration results       |
| <code>ldap_parser.py</code>          | LDAP query results            |
| <code>bloodhound_parser.py</code>    | BloodHound JSON data          |
| <code>delegation_parser.py</code>    | Delegation configuration data |
| <code>netexec_parser.py</code>       | NetExec output                |

## Collectors (6)

| Collector                            | Description                         |
|--------------------------------------|-------------------------------------|
| <code>bloodhound_collector.py</code> | BloodHound data collection          |
| <code>delegation_collector.py</code> | Delegation configuration collection |
| <code>dns_collector.py</code>        | DNS record collection               |
| <code>kerberos_collector.py</code>   | Kerberos-specific data collection   |
| <code>ldap_collector.py</code>       | LDAP enumeration                    |
| <code>smb_collector.py</code>        | SMB share and session enumeration   |

## Analyzers (5)

| Analyzer                                  | Description                                              |
|-------------------------------------------|----------------------------------------------------------|
| <code>misconfiguration_analyzer.py</code> | AD misconfiguration detection (largest file: ~241 lines) |
| <code>permission_analyzer.py</code>       | ACL and permission analysis                              |
| <code>privilege_analyzer.py</code>        | Privilege escalation path analysis                       |
| <code>relationship_analyzer.py</code>     | Trust and group relationship analysis                    |
| <code>trust_analyzer.py</code>            | Domain/forest trust analysis                             |

## Attack Path Generators (6)

| Generator                                  | Description                   |
|--------------------------------------------|-------------------------------|
| <code>acl_paths.py</code>                  | ACL-based attack paths        |
| <code>asrep_paths.py</code>                | AS-REP roasting attack paths  |
| <code>delegation_paths.py</code>           | Delegation abuse attack paths |
| <code>gpo_paths.py</code>                  | GPO-based attack paths        |
| <code>kerberoast_paths.py</code>           | Kerberoasting attack paths    |
| <code>privilege_escalation_paths.py</code> | Privilege escalation chains   |

## Reporting (6)

| Reporter                                    | Description                      |
|---------------------------------------------|----------------------------------|
| <code>ad_summary_reporter.py</code>         | Overall AD summary               |
| <code>attack_path_reporter.py</code>        | Attack path documentation        |
| <code>attack_surface_reporter.py</code>     | AD attack surface report         |
| <code>high_value_targets_reporter.py</code> | High-value target identification |
| <code>remediation_reporter.py</code>        | Remediation recommendations      |
| <code>report_builders.py</code>             | Report assembly utilities        |

## Phases (5)

| # | Internal<br>PHASE_NAME                 | CLI Slug                   | Key Actions                                                    |
|---|----------------------------------------|----------------------------|----------------------------------------------------------------|
| 1 | <code>passive_recon</code>             | <code>passive</code>       | DNS SRV enumeration, anonymous LDAP, SMB null sessions         |
| 2 | <code>identity_enumeration</code>      | <code>identity</code>      | User/group enumeration, RID cycling, Kerberos user enumeration |
| 3 | <code>configuration_enumeration</code> | <code>configuration</code> | GPO analysis, trust enumeration, misconfiguration detection    |
| 4 | <code>delegation_discovery</code>      | <code>delegation</code>    | Unconstrained/constrained/RBCD delegation detection            |
| 5 | <code>bloodhound_collection</code>     | <code>bloodhound</code>    | BloodHound data collection and graph analysis                  |

## Module-Specific Options

| CLI Flag                         | Description                                     |
|----------------------------------|-------------------------------------------------|
| <code>--domain corp.local</code> | AD domain name                                  |
| <code>-u / --username</code>     | Username for authenticated enumeration          |
| <code>-p / --password</code>     | Password for authenticated enumeration          |
| <code>--dc-ip</code>             | Domain Controller IP (if different from target) |

### `run()` Signature

```
ADModule.run(phases: Optional[List[str]] = None) -> Dict
```

## Workflow Orchestrator

The `WorkflowOrchestrator` ( `core/workflow_orchestrator.py` ) chains modules together:

### Pre-built Pipelines

**Full Recon** ( `WorkflowOrchestrator.full_recon()` ):

```
surface → network → ad (conditional) → web (conditional) → api (conditional)
```

**Targeted** ( `WorkflowOrchestrator.targeted()` ):

```
Runs specified modules sequentially without conditions.
```

### Conditional Branching

| Module               | Condition                                              |
|----------------------|--------------------------------------------------------|
| <code>surface</code> | Always runs if targets exist                           |
| <code>network</code> | Always runs (no condition)                             |
| <code>ad</code>      | Runs if LDAP, Kerberos, SMB, MSRPC, or domain detected |
| <code>web</code>     | Runs if HTTP/HTTPS service or URL detected             |
| <code>api</code>     | Runs if HTTP/HTTPS service or URL detected             |

## Shared Context

The `WorkflowContext` data bus carries discovered data between steps:

- `live_hosts`, `open_ports`, `services`, `domains`, `urls`
- Populated via `_extract_context_from_result()` after each module completes
- `CredentialVault` automatically shares credentials across modules

## CLI Usage

```
# Full recon (conditional branching)
python reconforge.py workflow --target 10.10.10.1

# Targeted modules
python reconforge.py workflow --target 10.10.10.1 --modules network,ad,web

# With engagement tracking
python reconforge.py workflow --target 10.10.10.1 --engagement "Q1 Pentest" --client "
Acme"

# Resume a paused engagement
python reconforge.py workflow --target 10.10.10.1 --resume /path/to/engagement.json
```